SOFTWARE TESTING AND QUALITY ASSURANCE (STQA)

LABORATORY PRACTICAL MANUAL

Practical 09

Implementing a Simple Continuous Testing Pipeline Using Jenkins

"To implement a simple continuous testing pipeline using a tool like Jenkins / Other open source software's (Github Co-pilot)."

Tools used	Python, PyTest, Git, GitHub, Jenkins
Class	B.E. Computer Science Engineering, Semester 7
Student Name	
Enrollment No.	
Date of Performance	
Faculty Signature	

Table of Contents

1. Practical Objective . . . 3
2. Basic Concepts . . . 3
3. Where Does the Test Actually Run? . . . 4
4. Tools Required . . . 5
5. Overall Architecture / Flow . . . 6
6. Sample Project . . . 6
7. Test the Project Locally First . . . 7
8. Create the GitHub Repository . . . 8
9. Install Jenkins Locally . . . 9
10. Understanding Jenkins . . . 9
11. Create the Jenkins Pipeline . . . 10
12. Explanation of Each Pipeline Stage . . . 11
13. Run the Jenkins Pipeline . . . 11
14. Demonstrate a Failed Test . . . 12
15. Fix the Code . . . 13
16. Continuous Testing Flow . . . 14
17. Manual Build vs Automatic Trigger . . . 14
18. GitHub Webhook (Optional, Advanced) . . . 15
19. Complete Practical Flow . . . 16
20. Important Clarification for Students . . . 16
21. Common Errors and Solutions . . . 17
22. Viva Questions . . . 17
23. Practical Output (Evidence to Submit) . . . 18
24. Final Conclusion . . . 18

1. Practical Objective

In real software projects, developers change code many times a day. Every change can accidentally break something that was working earlier. Testing everything by hand after every change is slow and unreliable. In this practical you will build a small system that **tests the software automatically every time the code changes**.

You will create a simple **continuous testing pipeline** in which:

1. Source code is stored in **GitHub**.
2. Automated test cases are created for the code.
3. **Jenkins** gets the latest code from GitHub.
4. Jenkins executes the automated test cases.
5. Jenkins displays whether the tests **passed or failed**.
6. When code is changed and pushed again, the pipeline can execute the tests again.

> **Purpose in one line**
> The purpose of this practical is to **automatically test software whenever code changes occur**, so that defects are found early and quickly.

2. Basic Concepts

Read these terms carefully. You do not need advanced DevOps knowledge for this practical; a simple understanding is enough.

Term	Simple Meaning
Continuous Testing	Running automated tests again and again, automatically, every time the code changes, so problems are detected immediately.
CI (Continuous Integration)	The practice of frequently merging code changes into a shared place and automatically building and testing them to check that everything still works together.
Automated Testing	Test cases written as code. A tool runs them and reports pass or fail, with no human clicking or checking.
Git	A **version control system** installed on your computer. It records every change made to your files so you can go back to older versions.
GitHub	A website that **stores Git repositories online**. It lets you keep your code safely on the internet and share it with others.
Jenkins	A free, open-source **automation server**. It automatically runs a series of tasks (get code, install, test) that you define.
Pipeline	A list of steps that Jenkins performs one after another, for example: Checkout, Install, Test.
Build	One complete run of the pipeline. Each run gets a number: Build #1, Build #2, and so on.
Test Execution	The actual running of the test cases by a testing framework (here, PyTest) to find out whether the code behaves as expected.

Who does what?

Tool	Role	Think of it as
Git	Tracks changes to your code (on your laptop)	A diary of every change
GitHub	Stores and manages the source code online	A cloud locker for code
Jenkins	Automatically executes the build and test process	A robot supervisor
PyTest / JUnit etc.	Testing framework that actually runs the test cases	The examiner who checks answers

> **Key idea**
> **GitHub** keeps the code. **Jenkins** triggers the process. **PyTest** does the testing. Jenkins does not know how to test your program by itself; it simply runs the testing tool for you.

3. Where Does the Test Actually Run?

This is a very important point. Jenkins is only a program, so **it must be installed and running on some machine**. The tests run on that machine. There are three possibilities:

Option	Jenkins is installed on	Typical use
A	The student's own laptop	Learning, practicals, personal projects
B	A college / company server	Shared team projects
C	A cloud server / virtual machine	Large or remote projects, always-on pipelines

> **Recommended for this practical: Option A**
> Installing Jenkins on your own laptop is the easiest, needs no extra account or server, and lets you see everything happening on your own machine.

Flow when Jenkins runs on the student's laptop

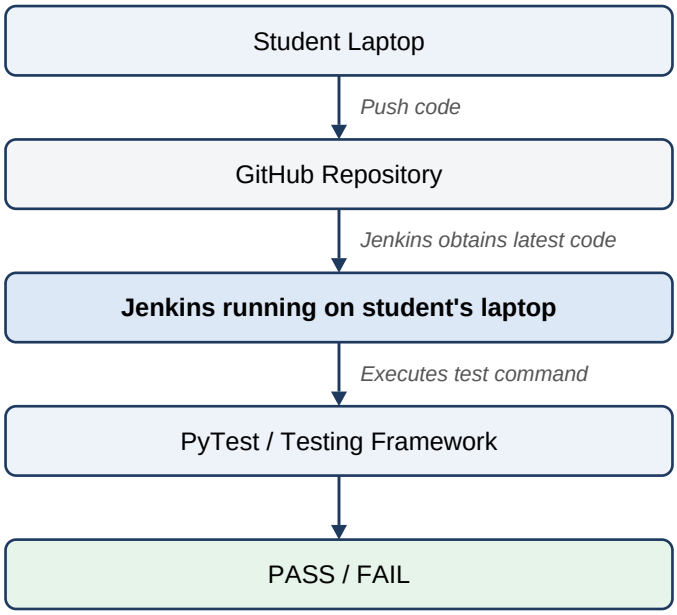

Figure 1: Where the code and the tests live in this practical

> **Remember**
> "The test cases are executed on the machine where Jenkins is running (or on a Jenkins agent configured for the job)."

Note about GitHub: GitHub is primarily used to **store the code**. GitHub does not automatically execute a Jenkins test by itself. Tests run automatically on GitHub only if another CI service such as **GitHub Actions** is used. In this practical, Jenkins is the one that runs the tests.

4. Tools Required

Tool	Purpose
Git	Version control; sends your code from your laptop to GitHub
GitHub	Online storage (repository) for the source code
Jenkins	Automation server that runs the pipeline
Java JDK	Jenkins is written in Java, so it needs Java to run (JDK 17 or 21 is recommended)
Python (3.x)	Programming language used for the sample project
PyTest	Python testing framework that runs the test cases
Web Browser	To open the Jenkins dashboard and GitHub
VS Code / any code editor	To write and edit the code

This practical uses **Python + PyTest + GitHub + Jenkins**. Students who prefer Java may use **Java + JUnit** instead, but the main practical in this manual is based on Python and PyTest.

5. Overall Architecture / Flow

The diagram shows the complete journey of a code change. The dashed boxes show *where* each step happens.

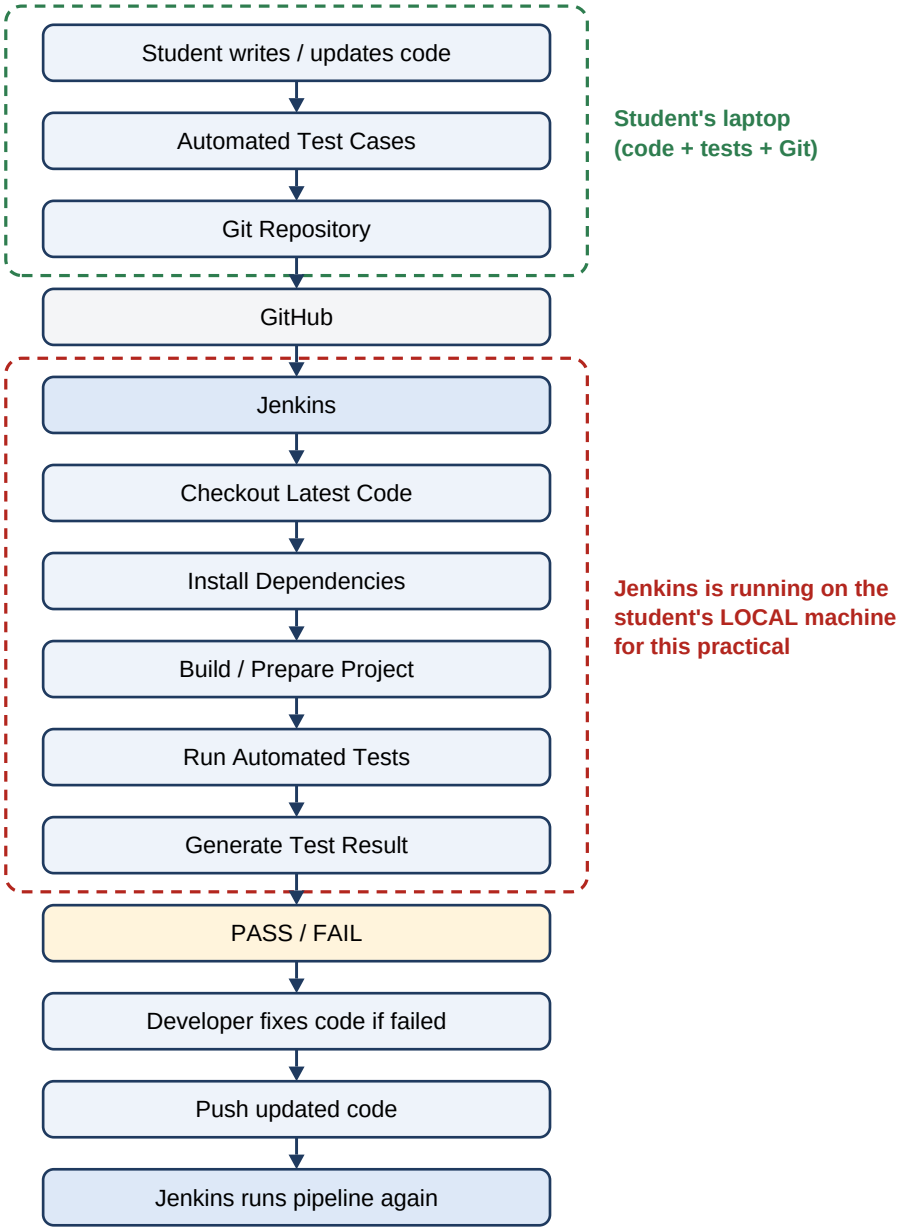

Figure 2: Overall architecture of the continuous testing pipeline

6. Sample Project

We will use a very simple Python **calculator**. Create a folder named `calculator-project` with the following three files.

```
calculator-project/
|
|-- calculator.py
|-- test_calculator.py
`-- requirements.txt
```

calculator.py

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    return a / b
```

test_calculator.py

```
from calculator import add, subtract, multiply, divide

def test_add():
    assert add(2, 3) == 5

def test_subtract():
    assert subtract(5, 3) == 2

def test_multiply():
    assert multiply(2, 4) == 8

def test_divide():
    assert divide(10, 2) == 5
```

requirements.txt

```
pytest
```

File	What it does
calculator.py	The application code. It contains four functions: add, subtract, multiply, divide.
test_calculator.py	The automated test cases. Each `test_` function calls one calculator function and uses `assert` to check that the result is what we expect. PyTest finds and runs every function whose name starts with `test_`.
requirements.txt	Lists the packages the project needs. Jenkins reads this file to install PyTest automatically.

7. Test the Project Locally First

Before using Jenkins, make sure the project works on your own computer. Open a terminal inside the `calculator-project` folder and run:

```
python --version
pip install pytest
pytest
```

(On Linux/macOS you may need to use `python3` and `pip3`.) The expected output is similar to:

Expected output

```
========== test session starts ==========
collected 4 items

test_calculator.py ....              [100%]

============ 4 passed in 0.05s ============
```

> **Why do this first?**
> First we make sure the application and test cases work correctly on the student's computer. Then Jenkins will **automate this same testing process**. If it does not work locally, it will not work in Jenkins either.

8. Create the GitHub Repository

Method 1: Using the GitHub website

1. Create a GitHub account at github.com, or log in if you already have one.
2. Click **New repository** (the green button, or the + icon at the top right).
3. Give it a suitable name, for example `stqa-continuous-testing`.
4. Choose **Public** (this keeps the practical simple because Jenkins will not need a password to download it).
5. Tick **Add a README file** if you want the repository to be initialised, then click **Create repository**.
6. Click **Add file > Upload files**, and upload `calculator.py`, `test_calculator.py` and `requirements.txt`. Click **Commit changes**.

Method 2: Using Git commands

Install Git first, then run these commands inside the project folder. Replace the URL with your own repository URL (copy it from the green **Code** button on GitHub).

```
git init
git add .
git commit -m "Initial project"
git branch -M main
git remote add origin <GitHub-Repository-URL>
git push -u origin main
```

Command	What it does
git init	Turns the current folder into a Git repository.
git add .	Selects all files in the folder to be saved in the next commit.
git commit -m "..."	Saves a snapshot of the selected files with a short message.
git branch -M main	Renames the current branch to **main**.
git remote add origin <URL>	Connects your local repository to the GitHub repository.
git push -u origin main	Uploads your commits to GitHub (branch main).

> **Tip**
> If Git asks for a password when pushing, GitHub requires a **Personal Access Token** instead of your account password. Sign-in through the browser prompt (Git Credential Manager) also works on most systems.

9. Install Jenkins Locally

Jenkins is written in Java, so Java must be installed first.

1. **Install a JDK** (version 17 or 21) from adoptium.net or oracle.com.
2. **Verify Java** is installed:

```
java -version
```

3. **Download and install Jenkins** from jenkins.io (use the Windows installer, or the LTS package for your OS).
4. **Start Jenkins.** On Windows the installer starts it automatically as a service. On Linux use `sudo systemctl start jenkins`.
5. **Open Jenkins in the browser** at the address below.

```
http://localhost:8080
```

The first time, Jenkins asks for an **Administrator password**. It is stored in a file; the exact path is shown on the screen (on Windows usually `C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword`, on Linux `/var/lib/jenkins/secrets/initialAdminPassword`). Then choose **Install suggested plugins** (this includes the Git and Pipeline plugins), create your admin user, and continue to the dashboard.

> **What is localhost?**
> **"localhost" means the Jenkins server is running on your own computer.** The number 8080 is the port Jenkins listens on. Jenkins is now acting as a local automation server.

10. Understanding Jenkins

Term	Beginner-friendly meaning
Jenkins Controller	The main Jenkins server you open in the browser. It stores jobs, schedules builds and shows results.
Agent / Executor	The machine (or slot) that actually carries out the work of a build. Think of the controller as the manager and the agent as the worker.
Job	A saved task in Jenkins, for example "STQA-Continuous-Testing". A job can be run many times.
Pipeline	A job type where the steps are written as a script (code) instead of clicking options.
Stage	A named section of a pipeline, such as Checkout or Run Tests. Jenkins shows each stage as a separate block.
Build	One execution of a job. Each build has a number (#1, #2, ...) and a result (SUCCESS or FAILURE).

For this basic practical, students run the job on **the same computer where Jenkins is installed**. The line `agent any` in the pipeline simply means "run on any available agent", which here is your own laptop.

11. Create the Jenkins Pipeline

1. Open Jenkins (`http://localhost:8080`) and log in.
2. Click **New Item** on the left menu.
3. Enter the job name: `STQA-Continuous-Testing`.
4. Select **Pipeline**.
5. Click **OK**.
6. The configuration page opens. Scroll down to the **Pipeline** section.
7. Under **Definition**, choose **Pipeline script** (for this basic practical we type the script directly in Jenkins; the other option, *Pipeline script from SCM*, reads a file named `Jenkinsfile` stored in GitHub).
8. Paste the script below, replace the repository URL, and click **Save**.

Pipeline script (Windows)

```
pipeline {
    agent any

    stages {

        stage('Checkout') {
            steps {
                git branch: 'main',
                    url: 'YOUR_GITHUB_REPOSITORY_URL'
            }
        }

        stage('Install Dependencies') {
            steps {
                bat 'pip install -r requirements.txt'
            }
        }

        stage('Run Tests') {
            steps {
                bat 'pytest'
            }
        }
    }
}
```

> **Replace the placeholder**
> You must replace `YOUR_GITHUB_REPOSITORY_URL` with your actual repository URL, for example `https://github.com/yourname/stqa-continuous-testing.git`.

Linux / macOS: use `sh` instead of `bat`. On newer Linux systems a virtual environment is often needed for pip, so the two stages can be written as:

Stages for Linux / macOS

```
        stage('Install Dependencies') {
            steps {
                sh 'python3 -m venv venv && . venv/bin/activate && pip install -r requirements.txt'
            }
        }

        stage('Run Tests') {
            steps {
                sh '. venv/bin/activate && pytest'
            }
        }
```

12. Explanation of Each Pipeline Stage

Stage	Purpose	What exactly happens
Checkout	Get the latest code	Jenkins downloads (clones) the latest source code from your GitHub repository into its workspace folder.
Install Dependencies	Prepare the environment	Jenkins reads requirements.txt and installs PyTest and any other required packages.
Run Tests	Execute the test cases	Jenkins runs the `pytest` command. If all tests pass the stage succeeds; if any test fails the stage and the build fail.

Jenkins decides pass or fail from the **exit code** of the command. PyTest returns 0 when all tests pass and a non-zero value when any test fails, and Jenkins marks the build accordingly.

13. Run the Jenkins Pipeline

1. Click **Save** after configuring the pipeline.
2. On the job page, click **Build Now**.
3. Under **Build History** (left side), click the build number, e.g. **#1**.
4. Click **Console Output**.
5. Observe the execution: you will see the code being downloaded, packages installed and the tests running.

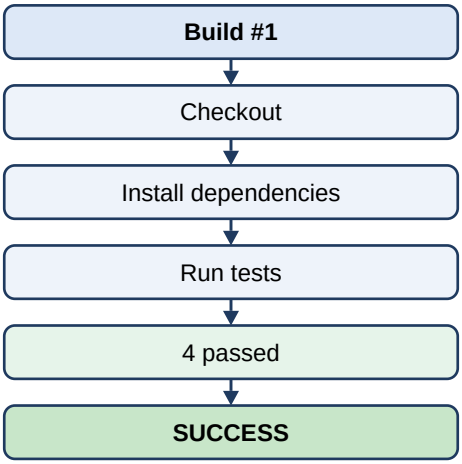

Figure 3: Flow of a successful build

> **What does SUCCESS mean?**
> It means every stage finished without error and all test cases passed. At the end of the Console Output Jenkins prints `Finished: SUCCESS` and shows a green/blue indicator. (Terms like "BUILD SUCCESS" or "BUILD FAILED" are used generally for the result; in a Jenkins pipeline the exact words are **SUCCESS** and **FAILURE**.)

14. Demonstrate a Failed Test

A test is only useful if it can detect a defect. Now **intentionally introduce an error** in `calculator.py`. Change:

```
def add(a, b):
    return a + b
```

to:

```
def add(a, b):
    return a - b
```

Commit and push the change to GitHub, then click **Build Now** again in Jenkins:

```
git add calculator.py
git commit -m "Introduce defect in add()"
git push
```

Expected result in the Console Output:

Expected output (approximate)

```
test_calculator.py F...                                  [100%]

FAILED test_calculator.py::test_add - assert -1 == 5
=========== 1 failed, 3 passed ===========
Finished: FAILURE
```

	Value
Expected	2 + 3 = **5**
Actual (with defect)	2 - 3 = **-1**

Why does it fail?

The test `test_add` expects `add(2, 3)` to return 5, but the faulty code returns -1. Expected and actual results do not match, so the automated test fails, and Jenkins reports **FAILURE** (shown in red). The defect was found automatically, without anyone checking by hand.

15. Fix the Code

Change the code back to the correct version:

```
def add(a, b):
    return a + b
```

```
git add calculator.py
git commit -m "Fix add() function"
git push
```

Click **Build Now** again. Expected result: **4 passed** and **SUCCESS**.

The basic idea of continuous testing

Code Change -> Automated Pipeline -> Automated Tests -> Result

16. Continuous Testing Flow

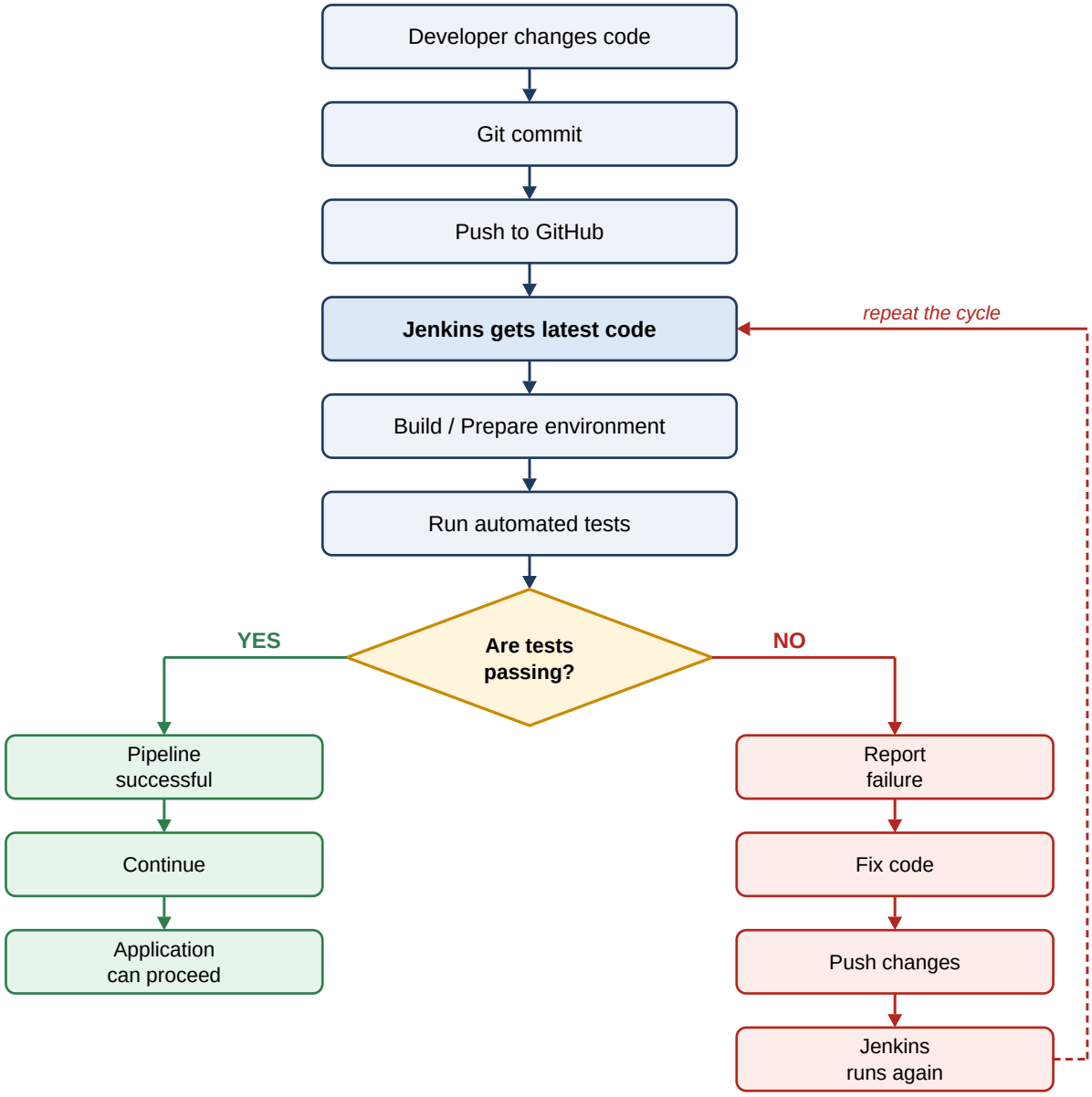

Figure 4: Continuous testing flowchart (the failure path loops back to Jenkins)

In a more advanced setup, a **GitHub webhook** can automatically trigger Jenkins whenever code is pushed. For this beginner practical, students may simply click **Build Now** manually if webhook configuration is not required.

17. Manual Build vs Automatic Trigger

	Manual trigger	Automatic trigger
Steps	Student pushes code, opens Jenkins, clicks **Build Now**	Student pushes code, GitHub webhook notifies Jenkins, Jenkins **starts the pipeline by itself**
Effort	One extra click every time	No extra action after setup
Setup	None	Webhook + Jenkins reachable from the internet
Use	Learning and practicals	Real CI workflow

Both methods demonstrate the pipeline, but webhook-based execution is **closer to a real continuous integration workflow**.

18. GitHub Webhook (Optional, Advanced)

> **Optional section**
> This part is **not mandatory**. Complete the basic manual-trigger practical first.

A webhook is a message that GitHub automatically sends to a web address whenever something happens, such as a push. If that address belongs to Jenkins, Jenkins starts the pipeline.

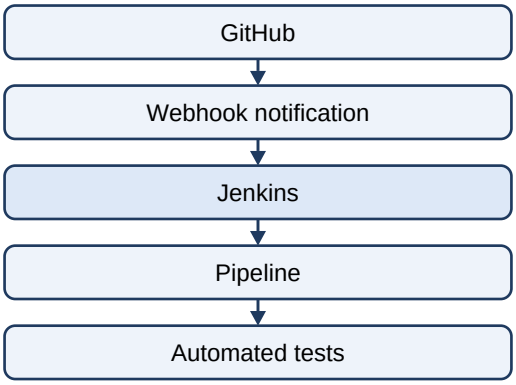

Figure 5: Webhook-triggered pipeline

Important: a Jenkins server on `localhost` cannot be reached by GitHub from the internet, so extra network configuration (such as a tunnelling tool or a public server) is needed. Students can therefore perform the basic manual trigger first. As a simpler alternative, Jenkins can check GitHub on a schedule using **Build Triggers > Poll SCM**.

19. Complete Practical Flow

A one-page summary of the whole practical. Tick each step as you complete it.

Phase	No.	Step	Done
Prepare	1	Create application	[ ]
	2	Create automated test cases	[ ]
	3	Test locally	[ ]
GitHub	4	Create GitHub repository	[ ]
	5	Push project to GitHub	[ ]
Jenkins setup	6	Install Java	[ ]
	7	Install Jenkins	[ ]
	8	Open Jenkins	[ ]
	9	Create Pipeline job	[ ]
	10	Connect Jenkins with GitHub	[ ]
Run pipeline	11	Checkout code	[ ]
	12	Install dependencies	[ ]
	13	Run automated tests	[ ]
	14	View test results	[ ]
Defect demo	15	Introduce a defect	[ ]
	16	Run pipeline	[ ]
	17	Observe failure	[ ]
	18	Fix defect	[ ]
	19	Push corrected code	[ ]
	20	Run pipeline again	[ ]
	21	Observe successful build	[ ]

20. Important Clarification for Students

> **REMEMBER**
>
> **GitHub** = Code Repository
>
> **Jenkins** = Automation / CI Server
>
> **PyTest** = Testing Framework
>
> **Test Case** = checks whether the expected behaviour is obtained
>
> **Jenkins does NOT itself perform the testing logic.** Jenkins executes the testing commands/tools such as PyTest, JUnit, Selenium, etc.

- If Jenkins is installed on **your laptop**, the tests execute on **your laptop**.
- If Jenkins is installed on a **college/company server**, the tests execute on **that server** (or its configured agent).

21. Common Errors and Solutions

Error	Likely cause	Solution
"java is not recognized"	Java/JDK is not installed or PATH is not configured	Install JDK, add its `bin` folder to PATH, reopen the terminal, run `java -version`
"python is not recognized"	Python is not installed or PATH is missing	Install Python and tick **Add Python to PATH** during installation
"pytest is not recognized"	PyTest is not installed	Run `pip install pytest` (or `python -m pytest`)
Jenkins cannot clone the GitHub repository	Wrong URL, wrong branch name, or private repository	Check the URL and branch (main); for private repos add credentials in Jenkins
Jenkins cannot run pip	Python is not installed or not visible to Jenkins	Check Python installation and PATH; **restart the Jenkins service** after changing PATH
Jenkins build fails	Any error in a stage	Open the build, click **Console Output** and read the first error message
Test fails	A defect in the code, or a wrong expected value in the test	Compare expected and actual values in the output; correct the application code or the test case

22. Viva Questions

Q1. What is continuous testing?

Ans: Automatically running tests every time the code changes, so defects are found early.

Q2. What is Jenkins?

Ans: A free, open-source automation server used to build, test and deploy software automatically.

Q3. What is CI?

Ans: Continuous Integration: frequently merging code into a shared repository and automatically building and testing it.

Q4. What is Git?

Ans: A version control system that tracks changes to files and lets you return to earlier versions.

Q5. What is GitHub?

Ans: An online platform that hosts Git repositories, used to store and share code.

Q6. What is the difference between Git and GitHub?

Ans: Git is the tool installed on your computer that tracks changes; GitHub is the website/service that stores Git repositories online.

Q7. Where does Jenkins execute the test cases?

Ans: On the machine where Jenkins is running (or on a Jenkins agent configured for the job). For this practical, it is the student's laptop.

Q8. What is a Jenkins pipeline?

Ans: A series of automated steps (such as checkout, install, test) defined as a script and run by Jenkins in order.

Q9. What is a Jenkins stage?

Ans: A named section of a pipeline that performs one part of the process, such as Checkout or Run Tests.

Q10. What is PyTest?

Ans: A Python testing framework that finds and runs test functions and reports whether they passed or failed.

Q11. Why do we use automated testing?

Ans: It is faster, repeatable, less error-prone than manual testing and can run on every code change.

Q12. What happens when a test fails?

Ans: The test command returns a non-zero exit code, the stage fails and Jenkins marks the build as FAILURE. The developer reads the output, fixes the code and pushes again.

Q13. What is the purpose of a GitHub webhook?

Ans: To automatically notify Jenkins when code is pushed so the pipeline starts without a manual click.

Q14. What is localhost?

Ans: A name that refers to your own computer; http://localhost:8080 means Jenkins is running on the same machine.

Q15. What is the difference between manual and automated testing?

Ans: In manual testing a person executes and checks each test; in automated testing test scripts are executed by a tool, quickly and repeatedly, with no human effort per run.

23. Practical Output (Evidence to Submit)

Capture the following screenshots and attach them to your journal or submission.

No.	Screenshot	Done
1	GitHub repository	[ ]
2	Project files (calculator.py, test_calculator.py, requirements.txt)	[ ]
3	Jenkins dashboard	[ ]
4	Jenkins Pipeline configuration	[ ]
5	Successful build	[ ]
6	Console Output showing tests passed	[ ]
7	Failed build after introducing the defect	[ ]
8	Successful build after fixing the defect	[ ]

24. Final Conclusion

Conclusion

The continuous testing pipeline successfully automates the process of obtaining source code from GitHub, installing dependencies, executing automated test cases, and reporting the test result through Jenkins. This demonstrates how automated testing can be integrated into a continuous integration workflow.